

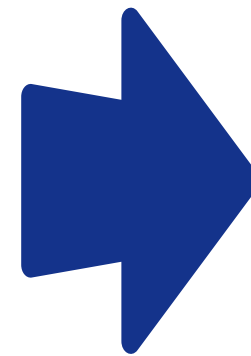
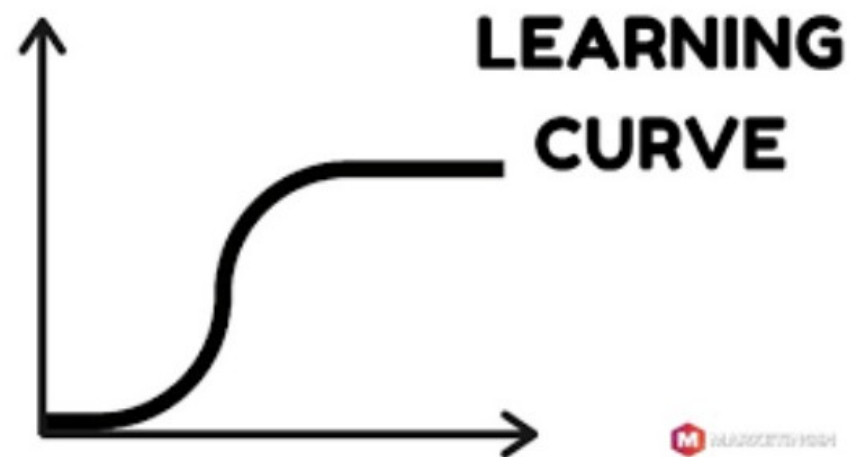
GUI 기반 FlowChart 에디터 개발 건의

04 플로우차트 GUI Editor 개발 건의

SURESOFT

Pain Point

현재 Flow Chart를 수정하려면,
⇒ Mermaid 코드를 직접 건드려서 수정해야함.



**GUI 기반으로 Flow Chart를 손쉽게
수정할 수 있는 기능의 필요성 대두**

코드 기반 수정이다보니, 사용자들에게 러닝커브 문제 존재

04 플로우차트 GUI Editor 개발 건의

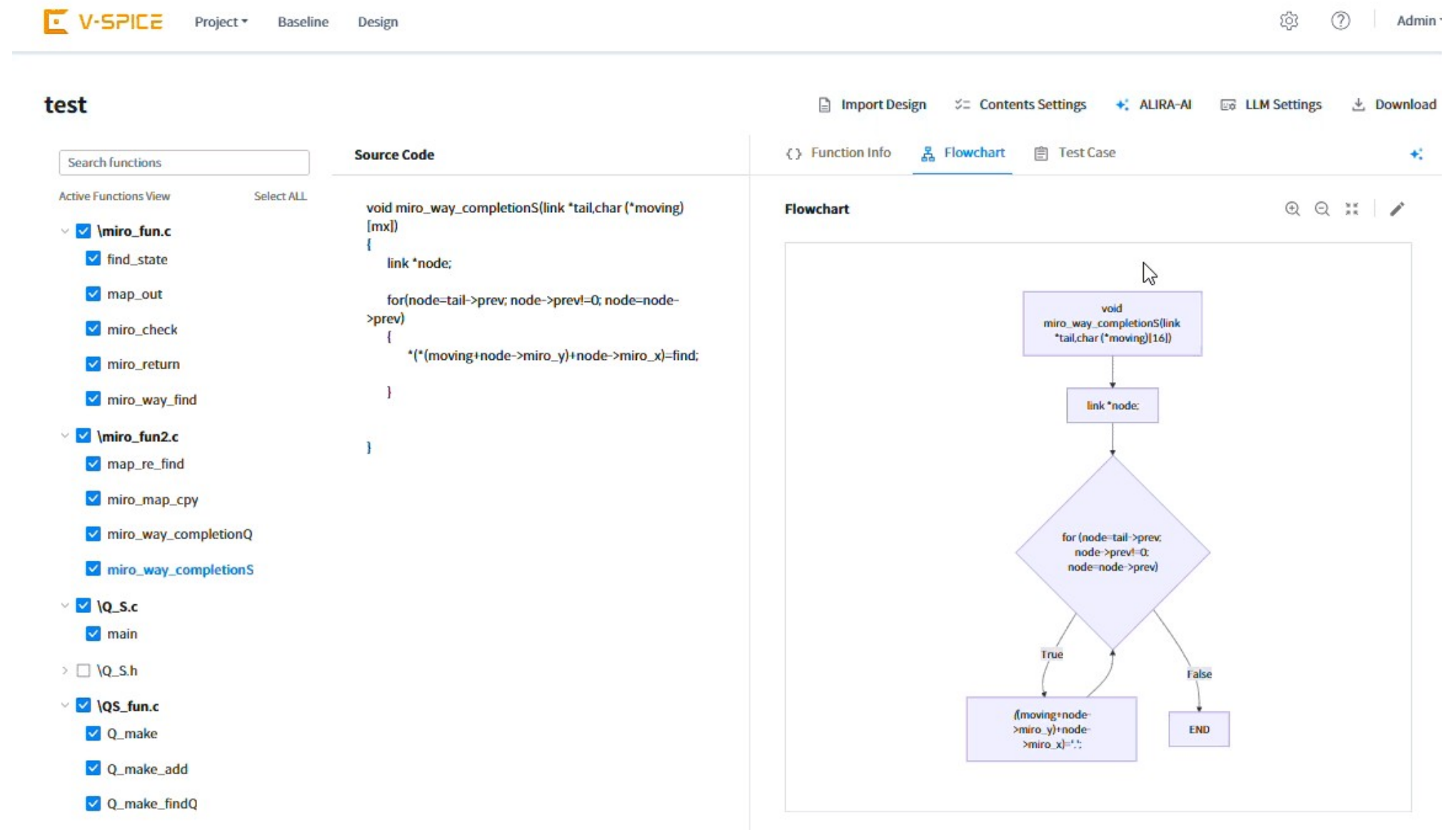
SURESOFT

그래서 일단 만들어보았다.

간단하게 프로토타입으로 개발 후
V-SPICE에 이식 후 작동 확인 완료



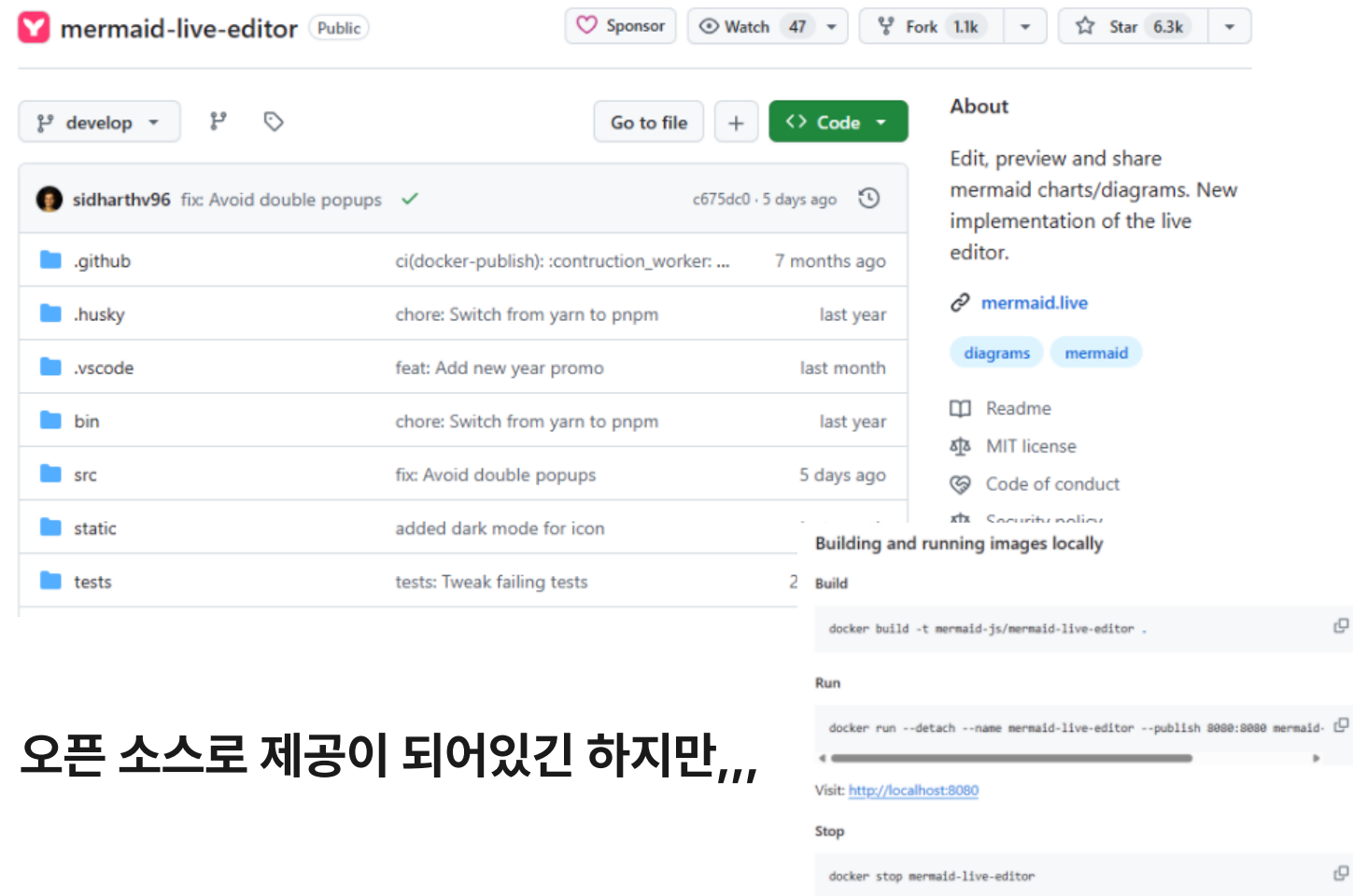
추가 고도화 진행 여부 검토 요청드립니다



04 플로우차트 GUI Editor 개발 건의

SURESOFT

Mermaid Live Editor가 도입 불가능한 이유



oopen 소스로 제공이 되어있긴 하지만,,,

```
docker build -t mermaid-js/mermaid-live-editor .

Run

docker run --detach --name mermaid-live-editor --publish 8080:8080 mermaid-

Visit: http://localhost:8080

Stop

docker stop mermaid-live-editor
```

Mermaid Live Editor는 라이브러리가 아닌 독립형 웹앱

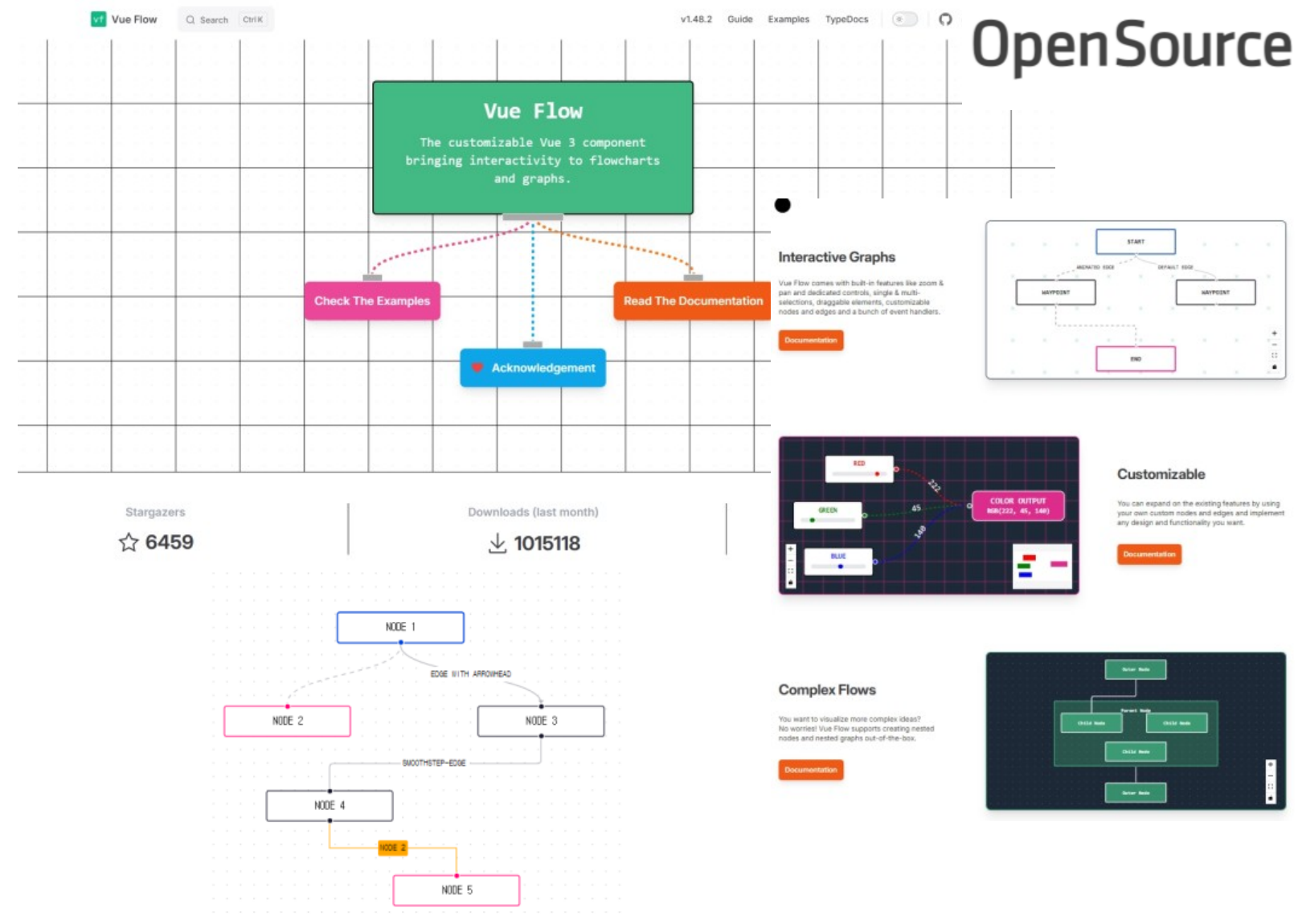


기존 화면에 직접 내장하기 어렵고,
Docker 기반 별도 실행 환경까지 필요해 도입 부담

04 플로우차트 GUI Editor 개발 건의

SURESOFT

Vue Flow 라이브러리



Vue Flow -

노드와 엣지로 구성된 다이어그램(플로우차트)을 직접 구현할 수 있는 UI 라이브러리

다이어그램을 구성하는 요소(노드, 연결선, 이벤트 등)를 제공하는 컴포넌트 형태의 도구

별도의 애플리케이션을 연동할 필요 없이,
기존 Vue 기반 서비스 내부에 컴포넌트 형태로
바로 삽입하여 사용 가능

04 플로우차트 GUI Editor 개발 건의

SURESOFT

Vue Flow 라이브러리

📖 README 📄 MIT license



Copyright (c) 2019-2025 webkid GmbH
Copyright (c) 2021-2025 Burak Cakmakoglu

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

MIT License

- 상업적 사용 가능 (판매 제품 포함)
- 수정 / 배포 / 재판매 모두 허용
- 단, 라이선스 문구 포함 필수

→ 상업적 제품 내장 및 판매가 가능하며, 라이선스 고지만 유지하면 자유롭게 사용 가능

04 플로우차트 GUI Editor 개발 건의

SURESOFT

Vue Flow의 문제점

1. Vue 3만 지원

🌱 Vue 2

This library doesn't work with Vue 2.

Vue Flow uses features that are exclusive to Vue 3, therefore there is no support for Vue 2, nor will there be any support in the future, sorry.

- Vue 3 전용 라이브러리
- V-SPICE는 Vue 2.6 기반 환경
- 공식적으로 Vue 2 지원 계획 없음

2. Vue Flow - Mermaid 공식 연동 기능의 부재

📄 Mermaid

```
graph TD
  A[Start] --> B[Process]
  B --> C{Decision}
  C -->|Yes| D[End]
  C -->|No| E[Retry]
```



📄 JavaScript

```
const nodes = [
  { id: '1', label: 'Start', position: { x: 0, y: 0 } },
  { id: '2', label: 'Process', position: { x: 200, y: 0 } },
  { id: '3', label: 'Decision', position: { x: 400, y: 0 } },
]

const edges = [
  { id: 'e1-2', source: '1', target: '2' },
  { id: 'e2-3', source: '2', target: '3' },
]
```

- 서로 쓰는 문법 다름
- Vue Flow는 Mermaid를 위한 라이브러리가 아니라,
⇒ 아예 다른 방식의 다이어그램 엔진
- 공식 플러그인 X

⇒ 그래서 크게 이 두 문제를 해결하면 도입이 가능하겠다 싶어서 해결방법 고민

04 플로우차트 GUI Editor 개발 건의

SURESOFT

Vue Flow 1번 해결안

절대 피해야하는 상황

에디터 하나 추가하려다 시스템 전체를 건드리게 된다.
에디터 기능 하나의 영향 범위가 시스템 전체로 번진다.
⇒ 런타임 충돌 및 전체 시스템 불안정

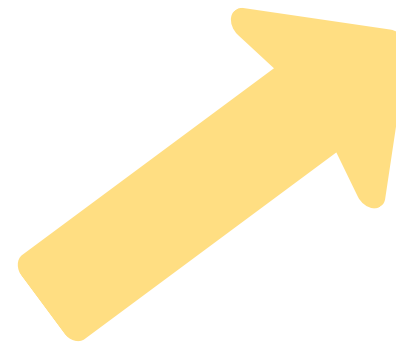


전제로 잡은 것

- 호스트 Vue 2 앱 코드 — 수정 없음
- 호스트의 빌드 설정, 패키지 의존성 — 변경 없음
- 배포 구조(Tomcat, WAR) — 변경 없음
- 두 런타임이 서로를 인식하지 못하도록 완전 격리

아이디어

에디터를 완전히 분리된 외부 프로젝트로 만들고,
빌드 결과물(정적 파일)만 호스트 앱에 넣는다.
⇒ 두 런타임의 완벽한 격리



따라서 이 문제의 해결 방향을 기술적 최적해보다 영향 범위 최소화를 1순위 기준으로 삼았다.

04 플로우차트 GUI Editor 개발 건의

SURESOFT

Vue Flow 1번 해결안



04 플로우차트 GUI Editor 개발 건의

SURESOFT

Vue Flow 1번 해결안

Why IIFE?

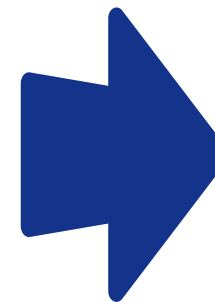
IIFE

(Immediately Invoked Function Expression)



다른 번들 포맷과의 비교

포맷	설명	문제
ESM	<code>import/export</code> 사용	모듈 시스템 필요, Vue 2 앱과 공존 어려움
CJS	<code>require()</code> 사용	브라우저에서 직접 사용 불가
UMD	ESM+CJS 혼합	복잡, <code>define / module / exports</code> 전역 참조
IIFE	즉시 실행 함수	전역 스코프 오염 없음, <code><script></code> 태그 하나로 로드 가능



IIFE 빌드 파일 하나 안에는

- Vue 3 런타임
- Vue Flow 컴포넌트
- 각종 직렬화 로직
등등..

모든 의존성을 포함한 즉시 실행 가능한 캡슐화 된 모듈



함수 스코프 안에서 작동하는 일종의 미니 도커 느낌

⇒ 외부엔 `window.MermaidFlowEditor`만 노출된 체로 격리

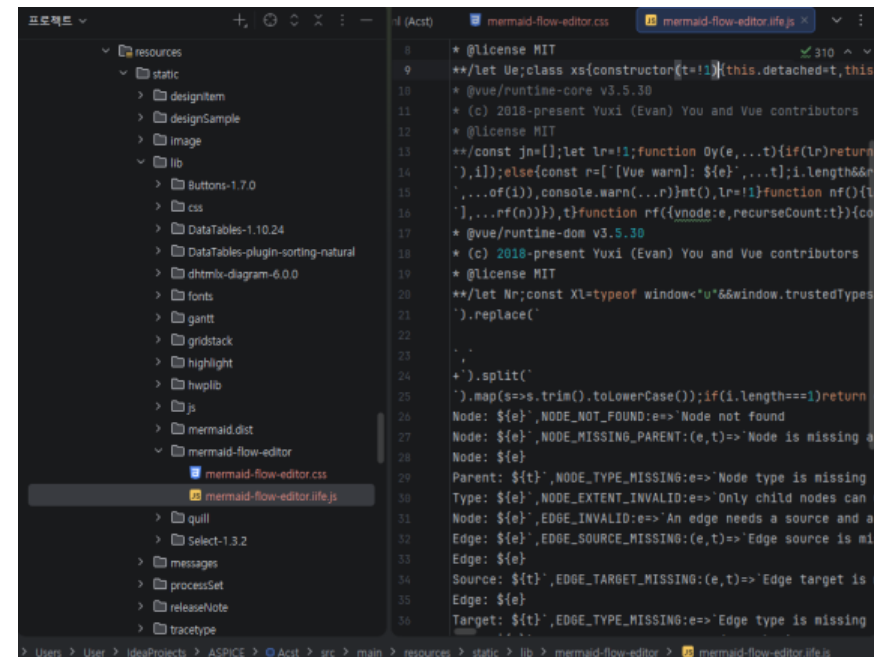
04 플로우차트 GUI Editor 개발 건의

SURESOFT

Vue Flow 1번 해결안

IIFE 이식

새로 생성된 파일 2개 (빌드 결과물)

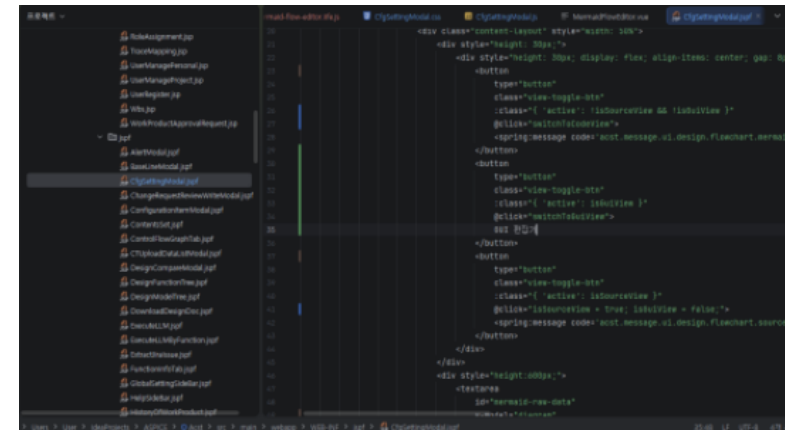


static/lib/mermaid-flow-editor/mermaid-flow-editor.iife.js
static/lib/mermaid-flow-editor/mermaid-flow-editor.css

외부 프로젝트에서 빌드된 결과물 - 정적 리소스로 저장

기존 파일 3개 수정

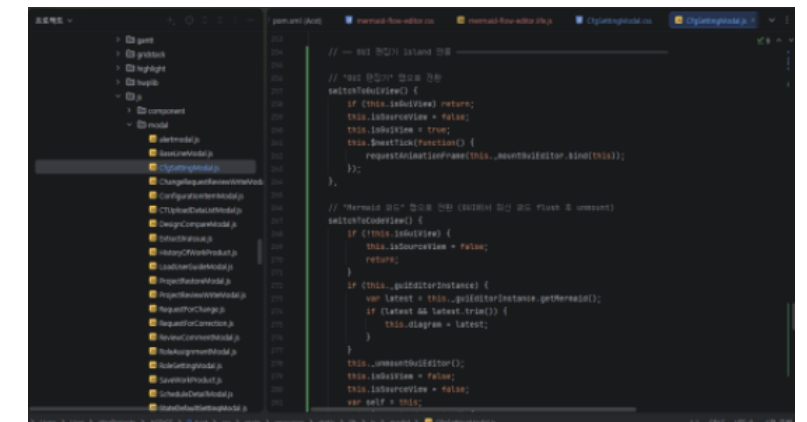
Acst/src/main/webapp/WEB-INF/jspf/CfgSettingModal.jspf



CfgSettingModal.jspf

HTML 빼대 - IIFE/CSS 로드

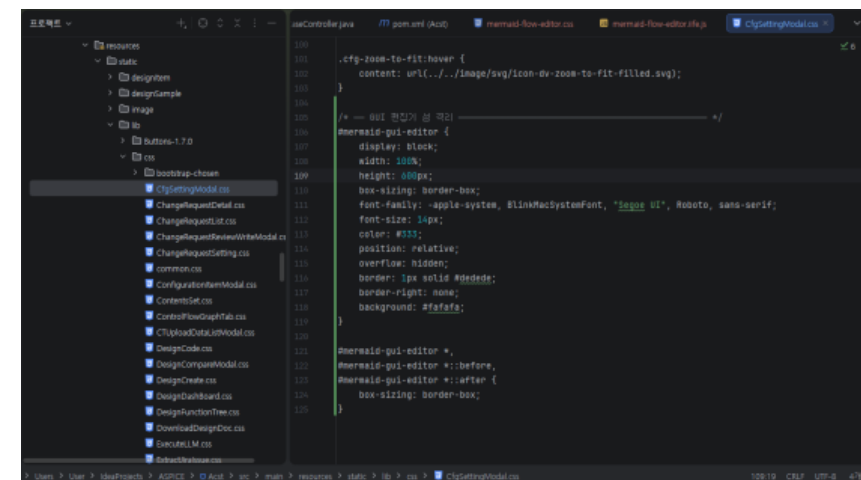
Acst/src/main/resources/static/lib/js/modal/CfgSettingModal.js



CfgSettingModal.js

Vue 2→3 제어 및 데이터 교환 로직

Acst/src/main/resources/static/lib/css/CfgSettingModal.css



CfgSettingModal.css

스타일 격리 - 호스트 CSS 침투 차단

04 플로우차트 GUI Editor 개발 건의

SURESOFT

Vue Flow 1번 해결안

호스트-에디터 통신 흐름 요약

[Vue 2 — CfgSettingModal.js]



[Vue 3 — IIFE 번들 내부]



switchToGuiView()

GUI 편집 시작시 IIFE와 마운트

MermaidFlowEditor.mount('#mermaid-editor', diagram)

함수 호출 방식

Vue 3 앱 생성 + 렌더링

changeHandler: self.diagram =
event.detail.mermaid
self.renderMermaid()

수정된 flow chart에 맞춰 mermaid 리렌더링

CustomEvent('mermaid-flow-editor:change', { mermaid })

DOM 이벤트 버스

사용자가 노드 편집

수정될때마다 수정된 flow chart code 이벤트 전송

switchToCodeView()

_guiEditorInstance.getMermaid()

GUI 편집 종료시 언마운트

MermaidFlowEditor._unmountGuiEditor()

CustomEvent('mermaid-flow-editor:change', { mermaid })

Vue 3 앱 정리 + DOM 복원

현재 mermaid 코드 반환

임무완수 - GUI Editor 컴포넌트 소멸

04 플로우차트 GUI Editor 개발 건의

SURESOFT

Vue Flow 1번 해결안

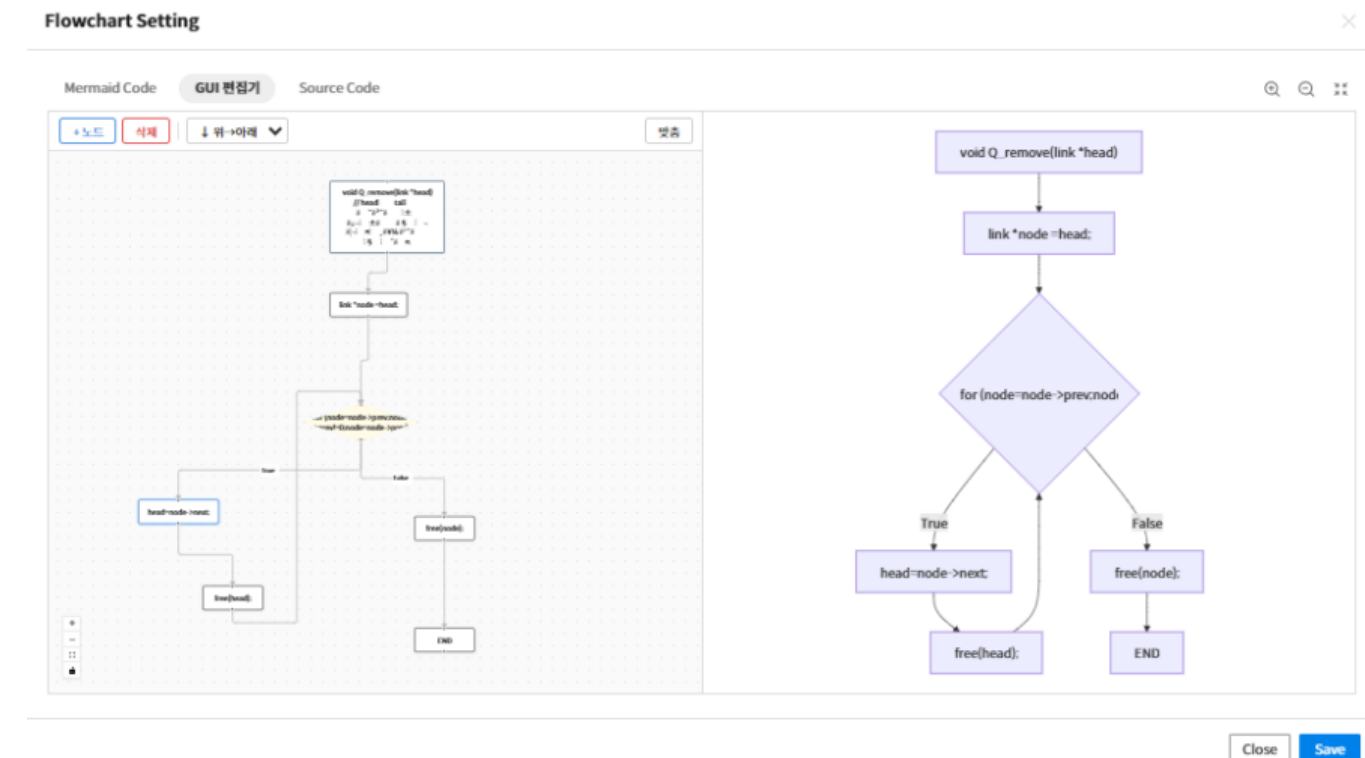
결과 및 결론

장점

- 외부와 스코프 격리 → Vue 2와 런타임 충돌 없음
- <script src> 한 줄로 로드 → 모듈 시스템 불필요
- 모든 의존성 내포 → 외부 CDN 없이 자급자족

단점

- 번들 크기 큼 → Vue 3 + VueFlow + D3 전부 포함, 수백 KB
- 격리가 스코프 레벨 → window, DOM은 여전히 공유.



IIFE 번들 기반 격리 및 이식 성공

04 플로우차트 GUI Editor 개발 건의

SURESOFT

Vue Flow 2번 해결안

문제점 2번

Mermaid

```
graph TD
  A[Start] --> B[Process]
  B --> C{Decision}
  C -->|Yes| D[End]
  C -->|No| E[Retry]
```



JavaScript

```
const nodes = [
  { id: '1', label: 'Start', position: { x: 0, y: 0 } },
  { id: '2', label: 'Process', position: { x: 200, y: 0 } },
  { id: '3', label: 'Decision', position: { x: 400, y: 0 } },
]

const edges = [
  { id: 'e1-2', source: '1', target: '2' },
  { id: 'e2-3', source: '2', target: '3' },
]
```

완전히 별개의 다이어그램 엔진이라
서로 데이터 포맷이 다르다.

하지만 Mermaid - Vue Flow 공식 플러그인 존재 **X**



아이디어

양방향 변환기를 직접 구현

- Mermaid Code To VueFlow 파싱
- VueFlow To Mermaid Code 직렬화

04 플로우차트 GUI Editor 개발 건의

SURESOFT

Vue Flow 2번 해결안

Mermaid ↔ VueFlow 손실 문제

서로 데이터 포맷이 달라서 지원 범위에 해당이 안될수도



Vue Flow 완전 지원 가능 범위

Mermaid	VueFlow 대응
노드 6종 shape	<code>node.data.shape</code>
노드 레이블	<code>node.data.label</code>
<code>--></code> 엣지	<code>edge.source / target</code>
<code>'--></code>	<code>label</code>
<code>flowchart TD/LR/BT/RL</code>	<code>direction</code> 상태값

Vue Flow 지원 범위 밖 기능들



Mermaid 기능	예시	안 되는 이유
노드 인라인 스타일	<code>style A fill:#f9f</code>	VueFlow 노드 객체에 스타일 필드 없음
클래스 정의	<code>classDef redBox fill:#f66</code>	VueFlow에 클래스 개념 없음
클래스 적용	<code>class A redBox</code>	동일
서브그래프	<code>subgraph 제목 ... end</code>	VueFlow에 그룹 개념 없음 (별도 구현 필요)
클릭 이벤트	<code>click A callback</code>	VueFlow 엣지/노드 데이터와 무관
주석	<code>% 주석</code>	데이터 구조에 주석 개념 없음
엣지 스타일	<code>linkStyle 0 stroke:#f66</code>	VueFlow edge에 인덱스 기반 스타일 없음

Mermaid → VueFlow → Mermaid 왕복 변환 시 지원 범위 밖 내용들은 손실 가능

04 플로우차트 GUI Editor 개발 건의

SURESOFIT

Vue Flow 2번 해결안

Mermaid ↔ VueFlow 손실 문제

Vue Flow 완전 지원 가능 범위

Mermaid	VueFlow 대응
노드 6종 shape	<code>node.data.shape</code>
노드 레이블	<code>node.data.label</code>
<code>--></code> 엣지	<code>edge.source / target</code>
<code>`--></code>	label
<code>flowchart TD/LR/BT/RL</code>	<code>direction</code> 상태값



ALIRA-AI

에서 출력하는 Mermaid 코드 수준

Flowchart Setting

Mermaid Code

GUI 편집기

Source Code

```
flowchart TD
  A["void miro_way_completionQ(link *tail,char (*moving)[16])"]
  B["link *node;"]
  C["for (node=tail->prev; node->prev!=0; node=node->prev)"]
  D["*(moving+node->miro_y)+node->miro_x='.';"]
  E["map_out(moving);"]
  F["END"]
  A --> B
  B --> C
  C -- True --> D
  D --> E
  E --> C
  C -- False --> F
```

ALIRA-AI의 출력 수준은 파서 완전 지원 범위 안에 완전히 포함된다.

⇒ 다행히도 실제 사용 시나리오에서 파싱 손실은 발생하지 않는다.

04 플로우차트 GUI Editor 개발 건의

SURESOFT

Vue Flow 2번 해결안

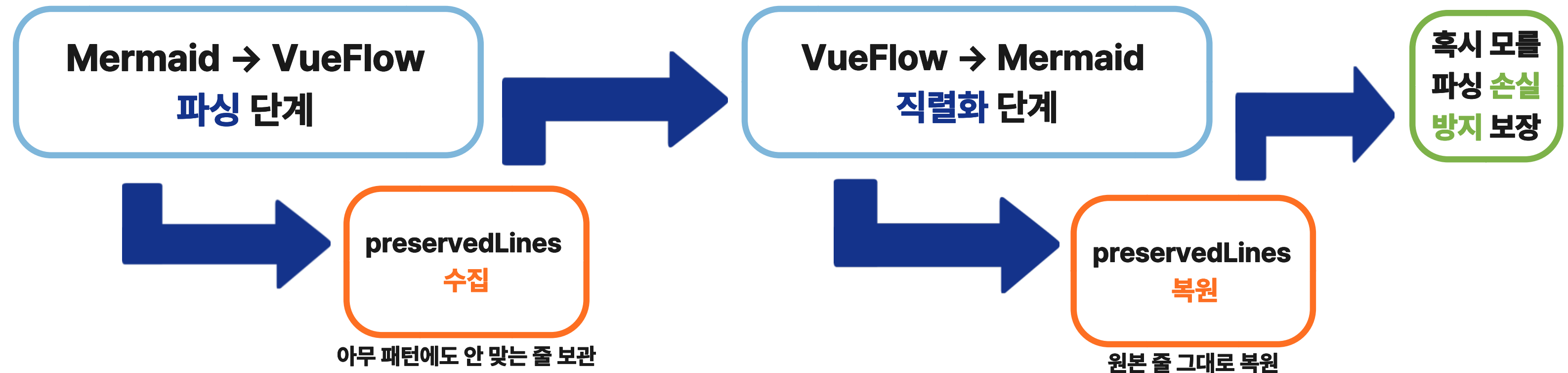
Mermaid ↔ VueFlow 손실 문제

그래도 혹시 모르니 안전 장치 추가

preservedLines

변환 불가 줄을 버리지 않는 안전망

파싱할 수 없는 줄을 버리지 않고
별도 배열에 그대로 보관한다.



Vue Flow 2번 해결안

Mermaid → VueFlow 파싱 (useMermaidParser.js)

```

flowchart TD
    A["A[\"void miro_way_completionQ(link *tail,char (*moving)[16])\"]"]
    B["B[\"link *node;\"]"]
    C["C[\"for (node=tail->prev; node->prev!=0; node=node->prev)\"]"]
    D["D[\"**(\"(moving+node->miro_y)+node->miro_x)=\".\",\")\"]"]
    E["E[\"map_out(moving);\"]"]
    F["F[\"END\"]"]
    A --> B
    B --> C
    C -- True --> D
    D --> E
    E --> C
    C -- False --> F

```

Mermaid Code가 입력으로 들어오면,

[illegible]

줄 단위로 순회하며 정규식 기반 추출 진행

```

flowchart TD
    A["void miro_way_completionQ(link *tail,char (*moving)[16])"] --> B["link *node;"]
    B --> C["for (node=tail->prev; node->prev!=0; node=node->prev)"]
    C --> D["**(*(moving+node->miro_y)+node->miro_x)='.';"]
    D --> E["map_out(moving);"]
    E --> F["END"]
    A --> B
    B --> C
    C -- True --> D
    D --> E
    E --> C
    C -- False --> F

```

→ direction 추출

→ 노드 파싱

→ 엣지 파싱 + 양끝 노드 추출

파싱 안된 줄들은 → preservedLines[] 에 보관

04 플로우차트 GUI Editor 개발 건의

SURESOFT

Vue Flow 2번 해결안

Mermaid → VueFlow 파싱
(useMermaidParser.js)

파싱 순회 끝,
nodesMap[] 및 rawEdges[] 완성



VueFlow는 position: { x, y }를 반드시 요구한다.
Mermaid Code엔 좌표 정보가 없으므로 직접 계산해야 함.

레이아웃 계산 로직

BFS 위상 정렬로 x/y 좌표 계산

1 인접 리스트 만들기

각 노드의 "부모"와 "자식"을 정리한다.

children (자식 목록)	parents (부모 목록)
A → [B, C]	A → []
B → [D]	B → [A]
C → [D]	C → [A]
D → []	D → [B, C]

2 BFS로 각 노드의 depth 계산

층별로 탐색 (루트: 부모 없는 노드)

```
const depth = {}
const queue = []
rawNodes.forEach(n => {
  if (parents[n.id].length === 0) {
    depth[n.id] = 0
    queue.push(n.id)
  }
})
```

```
const visited = new Set(queue)
let qi = 0
while (qi < queue.length) {
  const nid = queue[qi++]
  for (const child of children[nid]) {
    if (!visited.has(child)) {
      visited.add(child)
      depth[child] = depth[nid] + 1
      queue.push(child)
    }
  }
}
```

depth: { A:0, B:1, C:1, D:2 }

04 플로우차트 GUI Editor 개발 건의

SURESOFT

Vue Flow 2번 해결안

Mermaid → VueFlow 파싱
(useMermaidParser.js)

3 direction에 따라 x/y 축 방향 결정
TD 방향 기준 (depth = y축, 레이어 내 순서 = x축)

4 depth → 좌표 변환
같은 depth 노드들은 전체 너비 기준으로 중앙 정렬한다.

```
depth 0 → y=0 : A → x=중앙  
depth 1 → y=185 : B, C → x=왼쪽, x=오른쪽  
depth 2 → y=370 : D → x=중앙
```

결과

```
A: { x: 190, y: 0 }  
B: { x: 0, y: 185 }  
C: { x: 380, y: 185 }  
D: { x: 190, y: 370 }
```

node.position[]에 저장

레이아웃 계산 로직

BFS 위상 정렬로 x/y 좌표 계산

+ 사이클 문제 해결

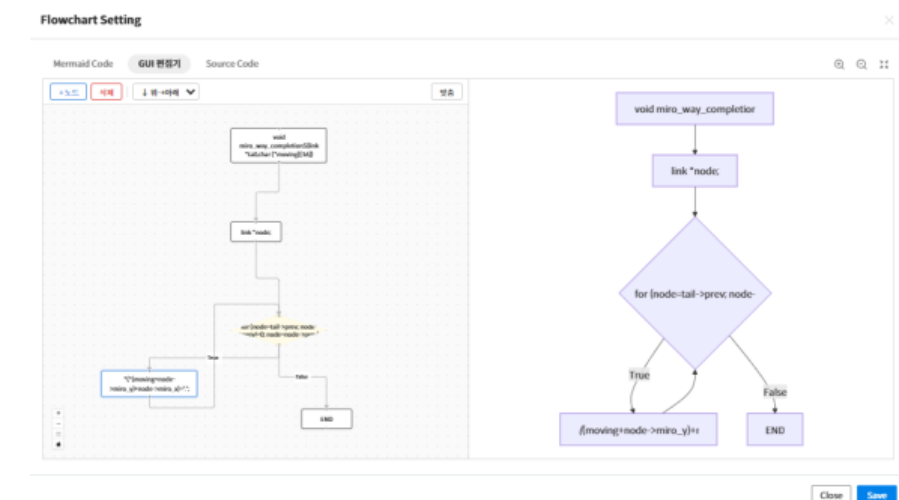
사이클 있는 FlowChart 그래프 파싱 시 무한루프 발생
뒤로가기 발생 → 무한루프 → 스택 오버플로 에러

SOLUTION

BFS에 visited Set 추가로 해결

이미 방문한 노드는 스킵 → back-edge 무시 → 정상 종료

잘 해결된 결과 →

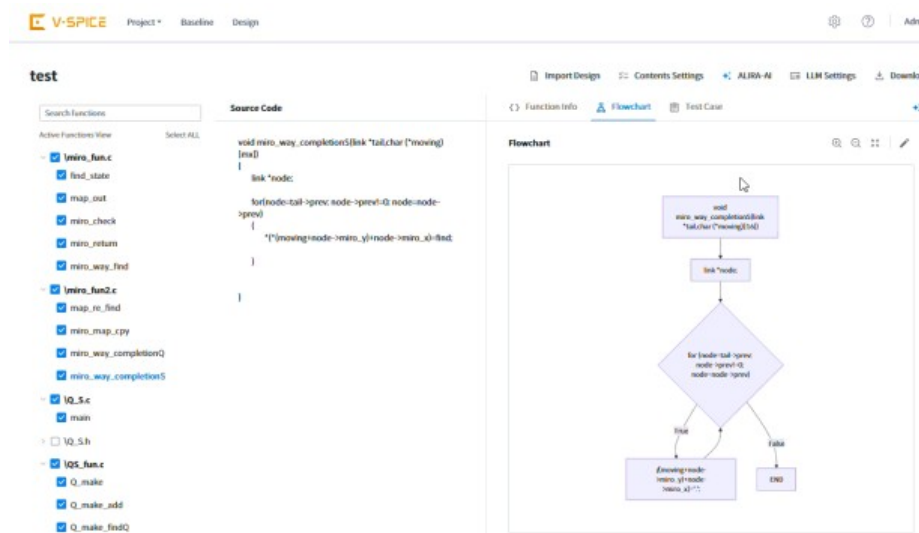


04 플로우차트 GUI Editor 개발 건의

SURESOFT

Vue Flow 2번 해결안

VueFlow → Mermaid 직렬화 (useFlowToMermaid.js)



Vue Flow에서 GUI 기반 플로우 차트 편집 후
다시 Mermaid Code로 변환하는 단계

Vue Flow 기준 데이터 포맷

```
nodes = [
  { id: 'A', data: { label: '시작', shape: 'stadium' } },
  { id: 'B', data: { label: '조건', shape: 'diamond' } },
  { id: 'C', data: { label: '실행', shape: 'default' } },
  { id: 'D', data: { label: 'END', shape: 'stadium' } },
]

edges = [
  { source: 'A', target: 'B', label: '' },
  { source: 'B', target: 'C', label: 'Yes' },
  { source: 'B', target: 'D', label: 'No' },
]

direction = 'TD'
preservedLines = []
```

Mermaid 문법으로 직렬화

- 1 헤더 출력
"flowchart {direction}" 헤더

04 플로우차트 GUI Editor 개발 건의

SURESOFT

Vue Flow 2번 해결안

VueFlow → Mermaid 직렬화 (useFlowToMermaid.js)

2 nodes 배열 순회

각 shape에 맞는 Mermaid 노드 문법 생성

```
/**
 * shape에 따라 Mermaid 노드 문자열 생성
 */
function nodeToMermaid(id, label, shape) {
  const l = sanitizeLabel(label || id)
  switch (shape) {
    case 'diamond': return ` ${id}{"${l}"}`
    case 'stadium': return ` ${id}(["${l}"])`
    case 'circle': return ` ${id}(("${l}"))`
    case 'round': return ` ${id}("${l}")`
    case 'parallelogram': return ` ${id}[/"${l}"/]`
    default: return ` ${id}["${l}"]`
  }
}
```

3 edges 배열 순회

레이블 유무에 따라 엣지 문법 생성

```
// 엣지 정의
edges.forEach(edge => {
  const src = edge.source
  const tgt = edge.target
  const label = edge.label || edge.data?.label || ''

  if (label) {
    lines.push(` ${src} -->| "${sanitizeLabel(label)}" | ${tgt}`)
  } else {
    lines.push(` ${src} --> ${tgt}`)
  }
})
```

04 플로우차트 GUI Editor 개발 건의

SURESOFT

Vue Flow 2번 해결안

VueFlow → Mermaid 직렬화
(useFlowToMermaid.js)

4 preservedLines 복원
(style, classDef 등)

```
// 보존된 줄 복원 (style, classDef, %% 등)
if (preservedLines && preservedLines.length > 0) {
  lines.push('')
  preservedLines.forEach(l => lines.push(l))
}
```

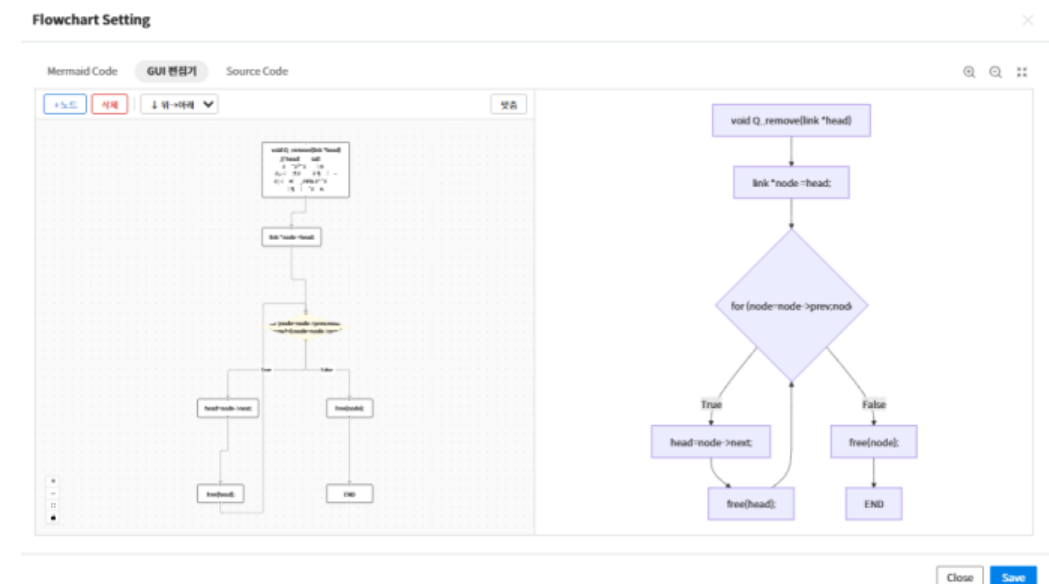


직렬화 결과

Mermaid Code 복원 완료

```
flowchart TD
  A(["시작"]) --> B["조건"]
  B --> C["실행"]
  C --> D(["END"])

  A --> B
  B -->|"Yes"| C
  B -->|"No"| D
```



04 플로우차트 GUI Editor 개발 건의

SURESOFT

Vue Flow 결론

현재 완성 상태

- 파싱/렌더링 핵심 기능
- GUI 편집 (노드 이동/추가/삭제/편집, 엣지 연결/삭제)
- Vue 2 호스트 연동 (CustomEvent, mount API)
- 편집 내용 → 코드 실시간 반영



꽤나 복잡한 그래프도 문제없이 동작

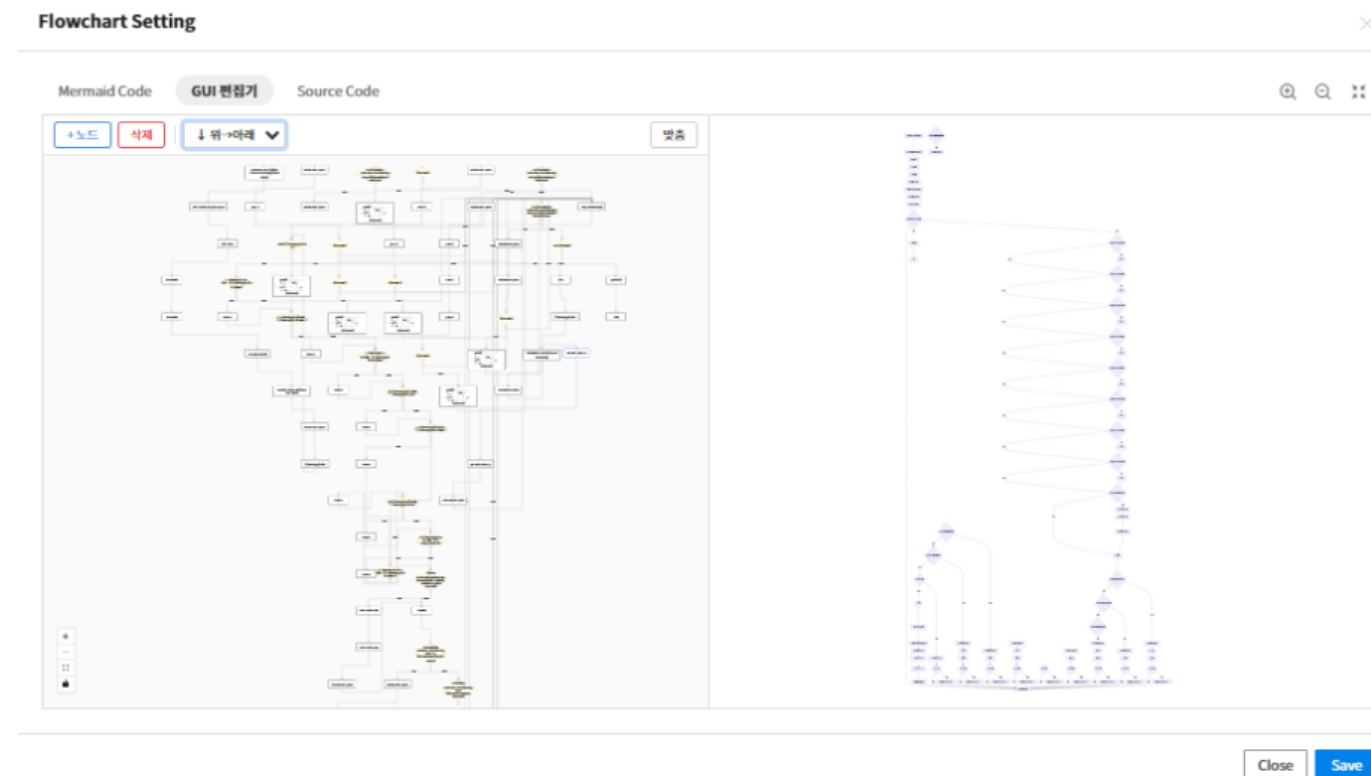
04 플로우차트 GUI Editor 개발 건의

SURESOFT

Vue Flow 결론

남은 과제들

- 파싱 정확도 개선
 - 방향 변경 시 레이아웃 재계산 추가
 - 다중 타겟 엣지 지원 추가
 - 노드 ID 재사용 충돌 가능성 개선
- UX/UI 개선
 - 엣지 레이블 편집 UI 추가
 - 사용성 및 디자인 개선
 - 파싱 실패 시 빈 캔버스 안내 추가



가장 큰 문제로는

- Flow Chart 규모가 커졌을때 헛갈릴 수 있게 배치되는것
⇒ 레이아웃 배치 계산 로직 고도화 필요

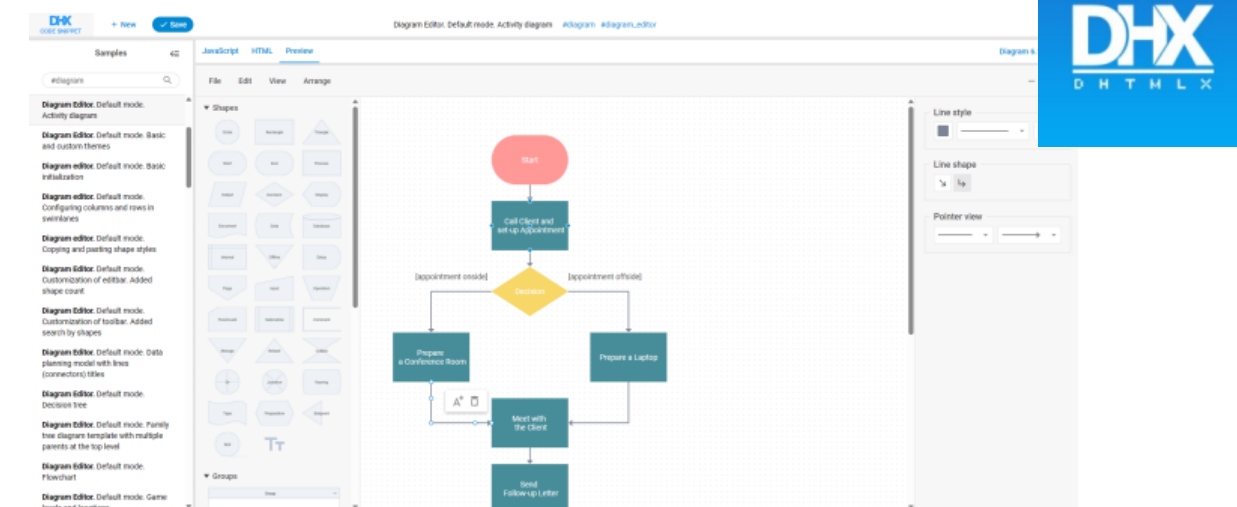
04 플로우차트 GUI Editor 개발 건의

SURESOFT

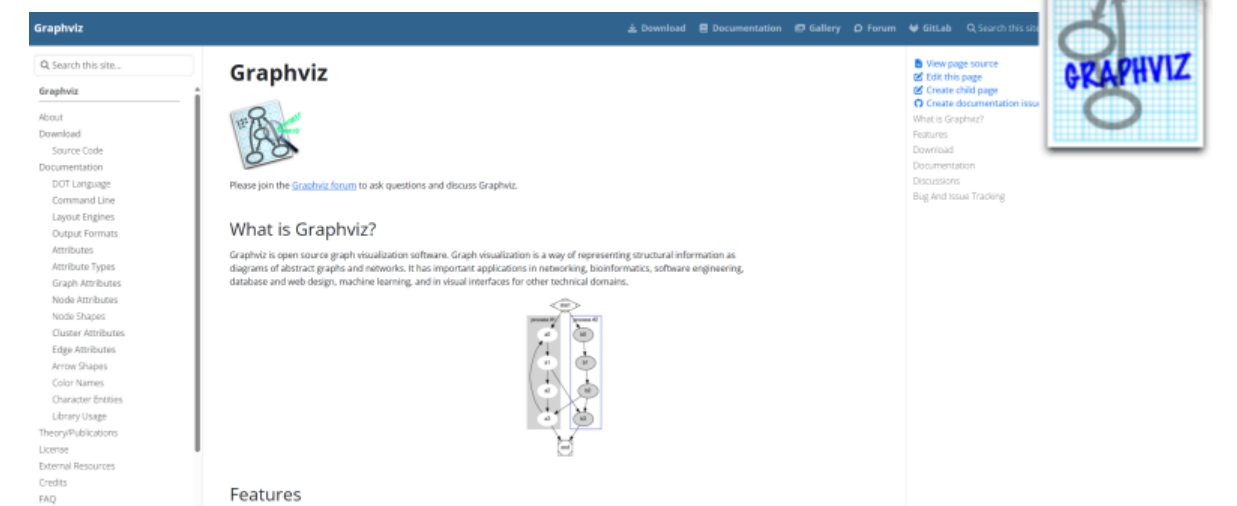
Vue Flow 결론

받은 피드백과 조언

- IIFE 삽입 시의 구조적 일관성 저하 및 유지보수 복잡성 문제
⇒ **DHTMLX** 라이브러리를 이용하는 쪽 추천
 - **Vue 2 환경에 바로 삽입 가능**
→복잡한 통신 제어 로직 및 IIFE 외부 빌드 과정 제거 가능
 - **다만 Mermaid 데이터 포맷과의 차이점 존재**
→Vue Flow 데이터 포맷과 유사해서 파싱/직렬화 로직 유지



- 레이아웃 배치(x/y) 계산 로직 고도화 필요
⇒ **Graphviz** 라이브러리 추천
 - 공식 문서 뜯어보면 배치 관련 알고리즘을 알수있지 않을까 싶음



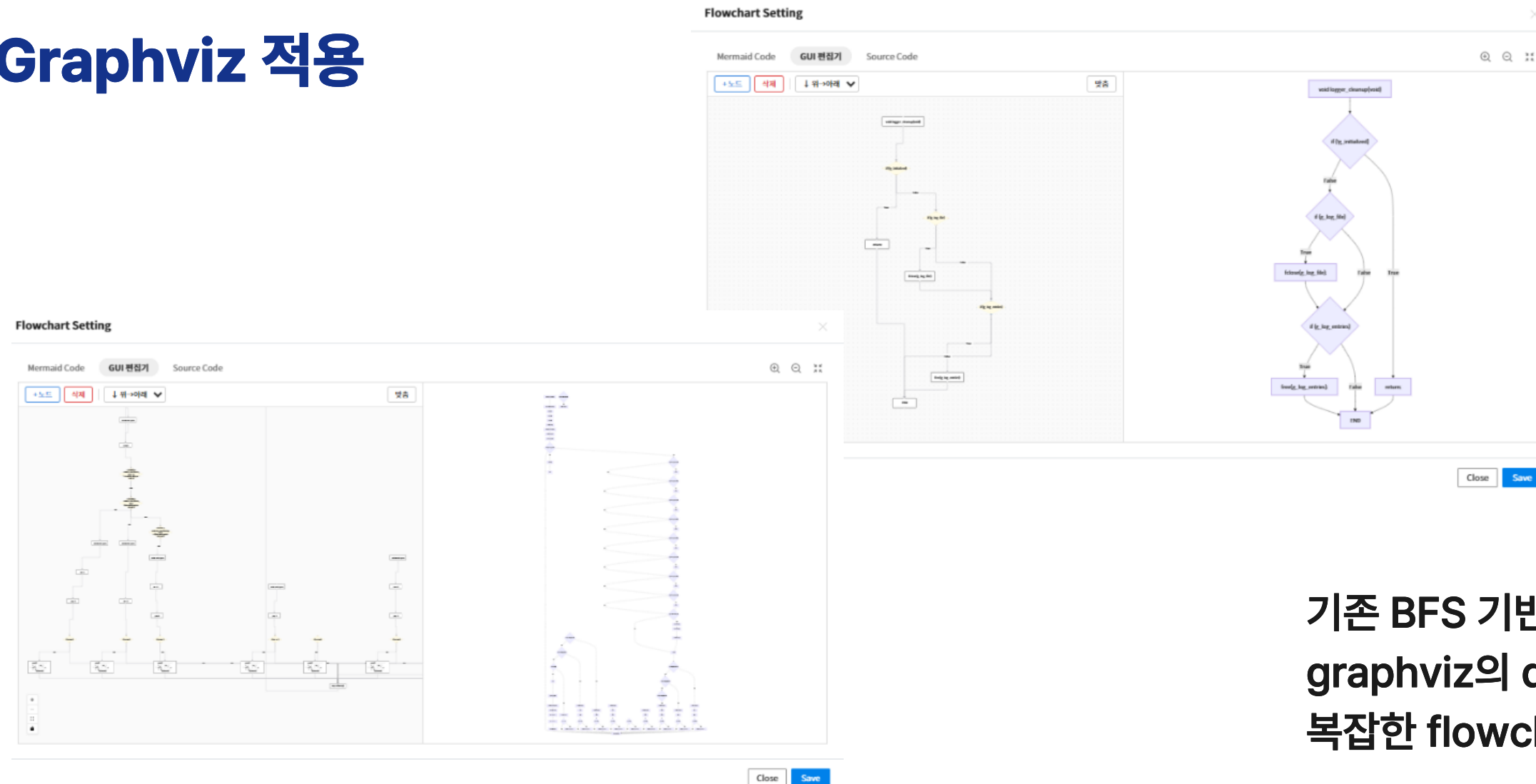
- **Sequence Diagram GUI Editor 기능도 필요**
→ 추후 SAD 기능에서도 필요한 핵심 기능
→ 이거까지 있어야 완전한 기능 제공 가능

04 플로우차트 GUI Editor 개발 건의

SURESOFT

Vue Flow 결론

Graphviz 적용



기존 BFS 기반 직접 계산 로직 대신
graphviz의 dot 엔진을 도입해서 레이아웃 계산
복잡한 flowchart에서도 꽤나 유효하게 바뀜

다만 완전 똑같진 않은 문제

04 플로우차트 GUI Editor 개발 건의

SURESOFT

Vue Flow 결론

DHTMLX 기반 구현

dagrejs/dagre

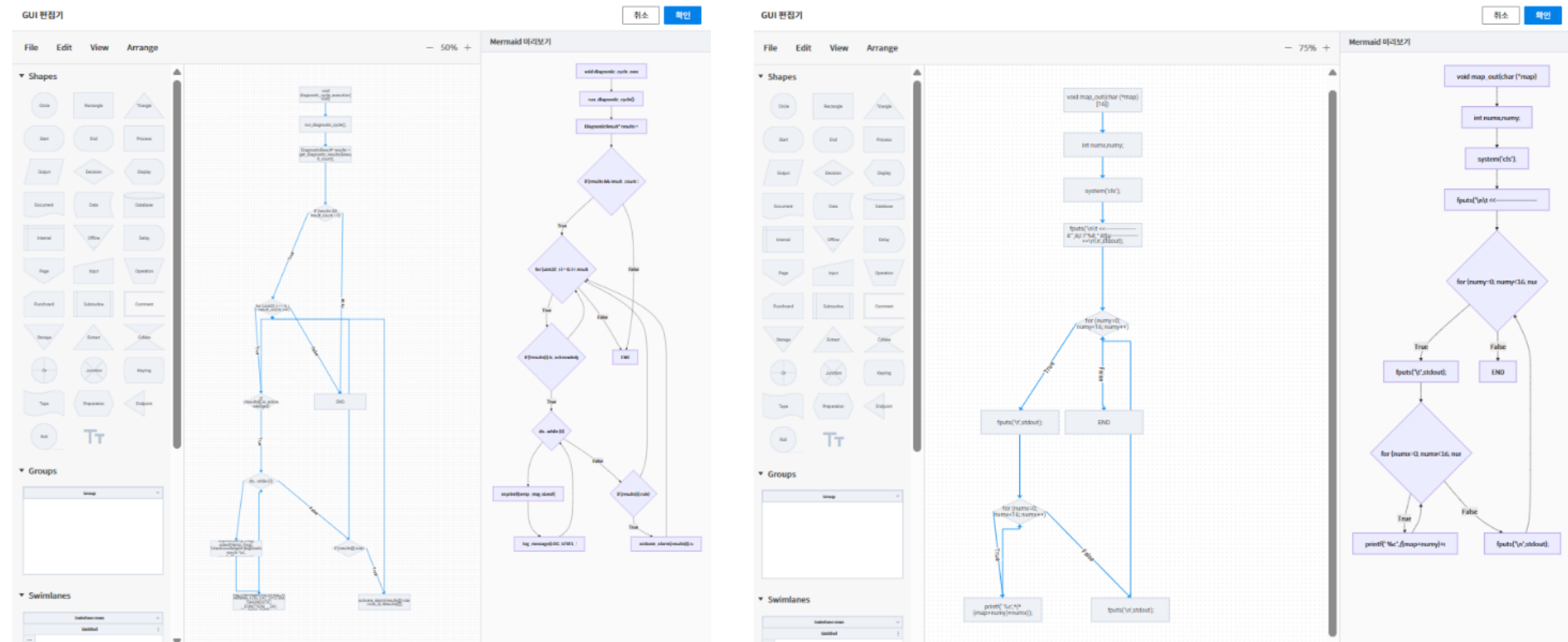
Directed graph layout for JavaScript



36 Contributors 62k Used by 6k Stars 639 Forks

실제 Mermaid에서 쓰는
레이아웃 계산 라이브러리 적용

본 프로젝트에 내장된 DHTMLX 라이브러리를 이용한 GUI 에디터로 교체한 모습



Mermaid 기반 svg Flowchart 노드 배치와 완벽하게 똑같이 배치 적용
하지만 Edge는 DHTMLX 에서 자동으로 배치를 만들기 때문에 이게 최선
추가로 DHTMLX에서 시퀀스 다이어그램을 공식 지원하지 않아 한계가 있었다.

04 플로우차트 GUI Editor 개발 건의

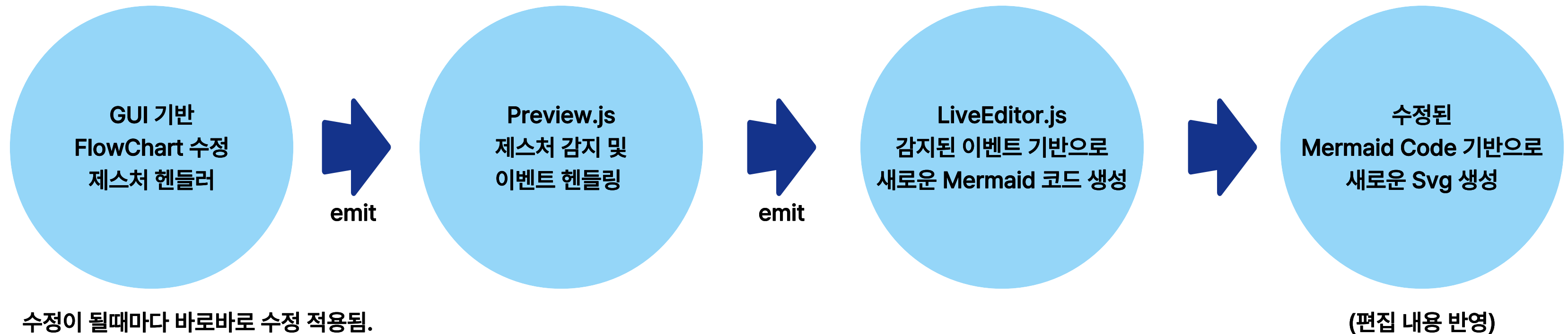
SURESOFT

Vue Flow 결론

직접 구현하는 방향

순수 JS 기반으로 직접 GUI Editor를 구현하는 방향

대략적인 작동 FLOW



04 플로우차트 GUI Editor 개발 건의

SURESOFT

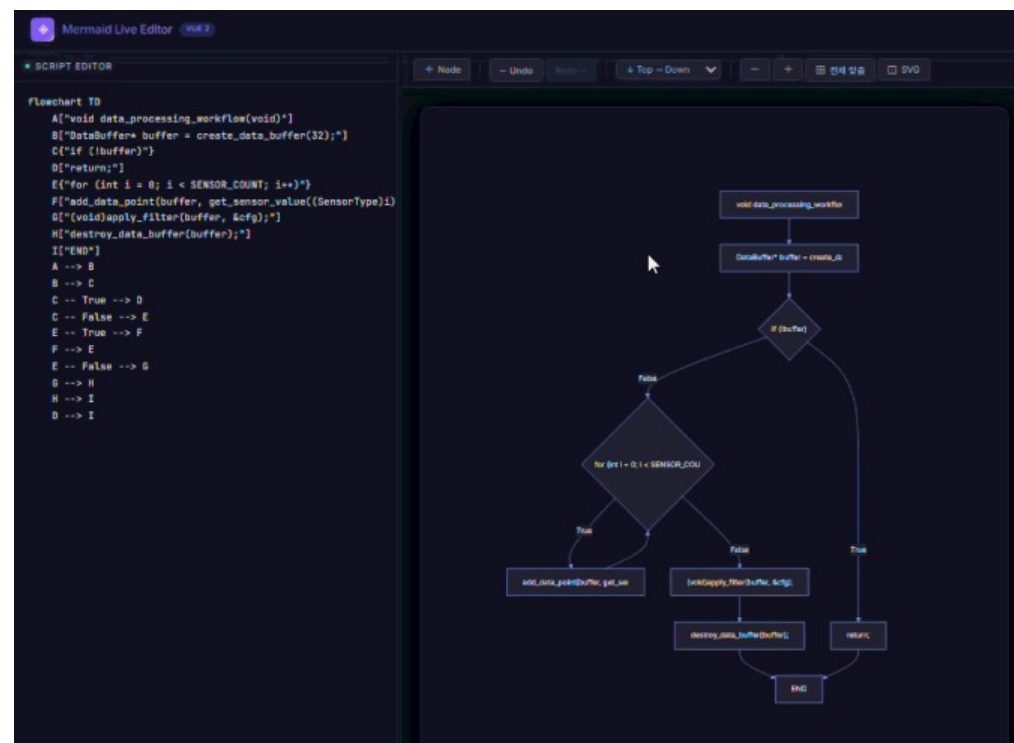
Vue Flow 결론

인계 받은 프로젝트 고도화

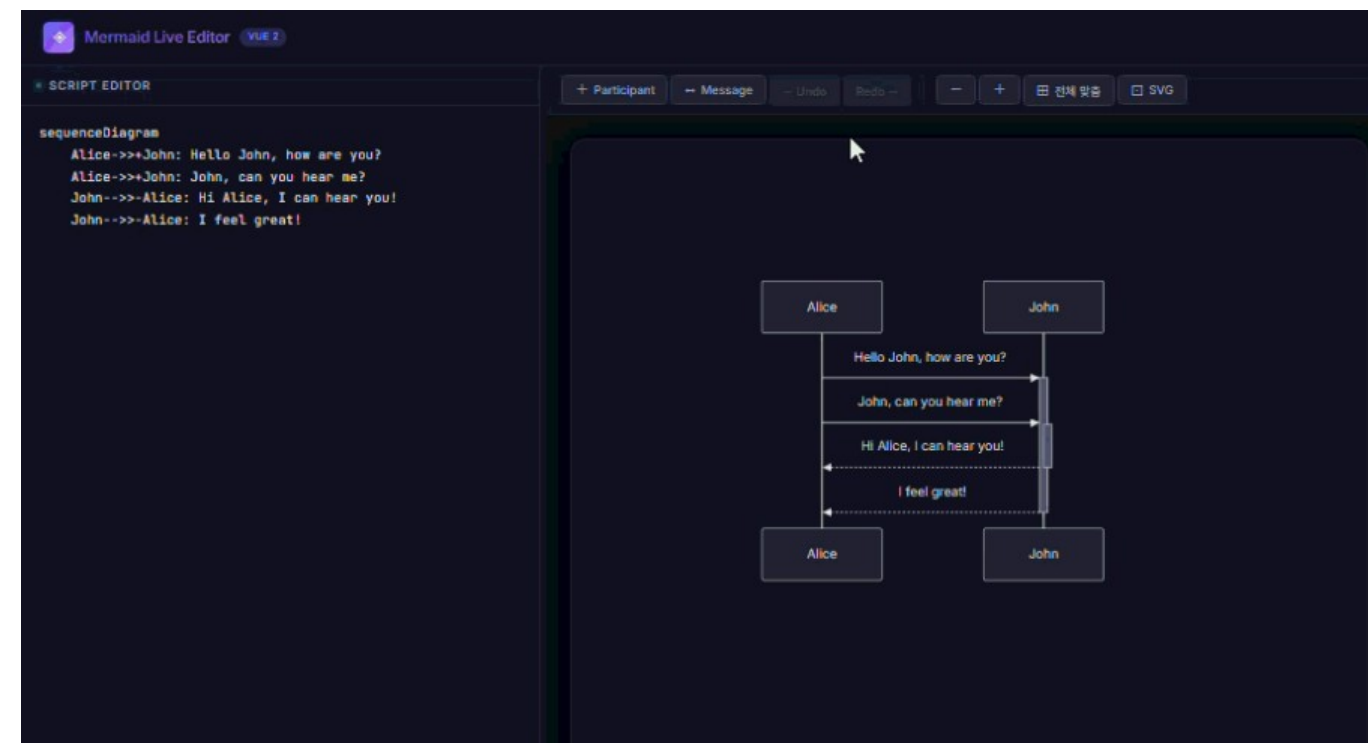
- 버그 수정 및 미구현 기능 구현
- 시퀀스 다이어그램 편집 기능 구현
- 사용성 개선

TODO

- 모든 기능 직접 구현이라 버그 꼼꼼히 검토 필요
- 사용성 고도화 필요
- State Diagram + Flow Diagram 대응 기능도 추가 요망



FlowChart 편집



Sequence Diagram 편집

감사합니다!